```
import os
import time
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC, LinearSVC
from sklearn.calibration import CalibratedClassifierCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    roc_auc_score, roc_curve, precision_recall_curve,
    average_precision_score, recall_score, f1_score
)
import shap


WESTERN_PATH = "shapfeatures.csv"
INDIAN_PATH  = "indian_AG.csv"
RESULTS_DIR  = "WestIndiSVM_fast"

os.makedirs(RESULTS_DIR, exist_ok=True)
```

```
RF_N_ESTIMATORS  = 200     # 200 is enough for feature ranking; was 400
SHAP_N_SAMPLES   = 200     # SHAP sample limit; was 500
TOP_N_GENES      = 20      # top SHAP genes for refined SVM; was 30
SVM_C_BASE       = 1.0
SVM_C_REFINED    = 10.0
CV_FOLDS         = 5
USE_OVERSAMPLING = False    # set True + uncomment import above if needed


def savefig(name):
    plt.savefig(os.path.join(RESULTS_DIR, name), dpi=150, bbox_inches='tight')
    plt.close('all')
    print(f"  → saved: {name}")

def timer(label):
    """Simple context-manager style timer."""
    class T:
        def __enter__(self):
            self.t = time.time()
            return self
        def __exit__(self, *a):
            print(f"  ⏱  {label}: {time.time()-self.t:.1f}s")
    return T()

sns.set_style("darkgrid")
BLUE, RED, GREEN, PURPLE, AMBER = '#4f8ef7', '#ef4444', '#10b981', '#7c3aed',
'#f59e0b'
```

```
# 1. LOAD DATA

print("=" * 65)
print("WestIndiSVM — Train: Western  |  Test: Indian  |  Model: SVM (fast)")
print("=" * 65)

def load_dataset(path, name):
    df = pd.read_csv(path).dropna()
    df = df.loc[:, ~df.columns.duplicated()]
    drop = ['Unnamed: 0', 'label', 'dataset_id', 'id', 'sample_id', 'target']
    feat_cols = [c for c in df.columns if c not in drop]
    feat_cols_clean = [c.strip().upper() for c in feat_cols]
    X = df[feat_cols].values.astype(np.float32)
    le = LabelEncoder()
    y = le.fit_transform(df['label'].values)
    print(f"\n{name}:  shape={X.shape}  tumor={( y==1).sum()}  normal={(y==0).sum()}")
    return X, y, feat_cols_clean

X_west, y_west, feat_west = load_dataset(WESTERN_PATH, "Western (train)")
X_indi, y_indi, feat_indi = load_dataset(INDIAN_PATH,  "Indian  (test)")


# 2. ALIGN FEATURES

west_map = {g: i for i, g in enumerate(feat_west)}
indi_map = {g: i for i, g in enumerate(feat_indi)}
common_genes = sorted(set(west_map) & set(indi_map))
```

```
print(f"\nCommon genes: {len(common_genes)}"
      f"  |  Western-only: {len(set(west_map)-set(indi_map))}"
      f"  |  Indian-only: {len(set(indi_map)-set(west_map))}")

if not common_genes:
    raise ValueError("No common genes — check column naming in both CSVs.")

west_idx = [west_map[g] for g in common_genes]
indi_idx = [indi_map[g] for g in common_genes]
X_west_al = X_west[:, west_idx]
X_indi_al = X_indi[:, indi_idx]
assert X_west_al.shape[1] == X_indi_al.shape[1]
print(f"✅ Aligned on {X_west_al.shape[1]} genes")

pd.DataFrame({
    'western_samples': [len(y_west)], 'indian_samples': [len(y_indi)],
    'common_genes': [len(common_genes)],
    'western_tumor': [(y_west==1).sum()], 'indian_tumor': [(y_indi==1).sum()],
}).to_csv(os.path.join(RESULTS_DIR, 'dataset_summary.csv'), index=False)

# ────────────────────────────────────────────────────────────────────────────
# 3. SCALE
# ────────────────────────────────────────────────────────────────────────────
scaler = StandardScaler()
X_west_sc = scaler.fit_transform(X_west_al)
X_indi_sc = scaler.transform(X_indi_al)


if USE_OVERSAMPLING:
```

```
    from imblearn.over_sampling import BorderlineSMOTE
    with timer("BorderlineSMOTE"):
        sampler = BorderlineSMOTE(random_state=42, k_neighbors=5)
        X_train, y_train = sampler.fit_resample(X_west_sc, y_west)
    print(f"  After oversampling — tumor: {(y_train==1).sum()}  normal:
{(y_train==0).sum()}")
else:
    X_train, y_train = X_west_sc, y_west
    print("\nOversampling: SKIPPED (using class_weight='balanced' instead)")

# ────────────────────────────────────────────────────────────────────────────
# 5. BASELINE SVM  — LinearSVC is 10-100x faster than SVC(kernel='linear')

print("\n[1/5] Baseline SVM (LinearSVC, all common genes)...")
with timer("Baseline SVM fit"):
    lsvc = LinearSVC(C=SVM_C_BASE, class_weight='balanced',
                     max_iter=2000, random_state=42)
    svm_base = CalibratedClassifierCV(lsvc, cv=3)
    svm_base.fit(X_train, y_train)

y_pred_base = svm_base.predict(X_indi_sc)
y_prob_base = svm_base.predict_proba(X_indi_sc)[:, 1]
print(f"  Accuracy: {accuracy_score(y_indi, y_pred_base):.4f}"
      f"  ROC-AUC: {roc_auc_score(y_indi, y_prob_base):.4f}")
print(classification_report(y_indi, y_pred_base, target_names=['Normal', 'Tumor']))

# ────────────────────────────────────────────────────────────────────────────
# 6. BIOMARKER DISCOVERY (Random Forest — reduced to 200 trees)
# ────────────────────────────────────────────────────────────────────────────
print(f"\n[2/5] Random Forest biomarker discovery ({RF_N_ESTIMATORS} trees)...")
```

```
with timer("RF fit"):
    rf = RandomForestClassifier(
        n_estimators=RF_N_ESTIMATORS, max_features='sqrt',
        class_weight='balanced', random_state=42, n_jobs=-1
    )
    rf.fit(X_train, y_train)

rf_importance = pd.Series(rf.feature_importances_,
index=common_genes).sort_values(ascending=False)
rf_importance.head(50).reset_index().rename(
    columns={'index': 'gene', 0: 'rf_importance'}
).to_csv(os.path.join(RESULTS_DIR, 'top50_RF_biomarkers.csv'), index=False)
print("  Top 10 RF biomarkers:")
print(rf_importance.head(10).to_string())

# ────────────────────────────────────────────────────────────────────────────
# 7. SHAP — sampled background + limited to SHAP_N_SAMPLES

print(f"\n[3/5] SHAP explainability ({SHAP_N_SAMPLES} Indian samples)...")
with timer("SHAP"):
    background  = shap.sample(X_train, 50, random_state=42)   # small background
    explainer   = shap.TreeExplainer(rf, background)
    shap_vals   = explainer.shap_values(X_indi_sc[:SHAP_N_SAMPLES])

    if isinstance(shap_vals, list):
        shap_tumor = shap_vals[1]
    elif shap_vals.ndim == 3:
        shap_tumor = shap_vals[:, :, 1]
    else:
        shap_tumor = shap_vals
```

```
shap_importance = pd.Series(
    np.abs(shap_tumor).mean(axis=0), index=common_genes
).sort_values(ascending=False)

shap_importance.head(50).reset_index().rename(
    columns={'index': 'gene', 0: 'mean_abs_shap'}
).to_csv(os.path.join(RESULTS_DIR, 'top50_SHAP_biomarkers.csv'), index=False)
print("  Top 10 SHAP biomarkers:")
print(shap_importance.head(10).to_string())

# ────────────────────────────────────────────────────────────────────────────
# 8. REFINED SVM — top-N SHAP genes, RBF kernel
# ────────────────────────────────────────────────────────────────────────────
print(f"\n[4/5] Refined SVM (RBF, top-{TOP_N_GENES} SHAP genes)...")
top_genes = shap_importance.head(TOP_N_GENES).index.tolist()
top_idx   = [common_genes.index(g) for g in top_genes]

X_train_top = X_train[:, top_idx]
X_indi_top  = X_indi_sc[:, top_idx]

with timer("Refined SVM fit"):
    svm_ref = SVC(kernel='rbf', C=SVM_C_REFINED, gamma='scale',
                  class_weight='balanced', probability=True, random_state=42)
    svm_ref.fit(X_train_top, y_train)

y_pred_ref = svm_ref.predict(X_indi_top)
y_prob_ref = svm_ref.predict_proba(X_indi_top)[:, 1]
print(f"  Accuracy: {accuracy_score(y_indi, y_pred_ref):.4f}"
```

```
    f"  ROC-AUC: {roc_auc_score(y_indi, y_prob_ref):.4f}")
print(classification_report(y_indi, y_pred_ref, target_names=['Normal', 'Tumor']))

# ────────────────────────────────────────────────────────────────────────────
# 9. CROSS-VALIDATION on Western
# ────────────────────────────────────────────────────────────────────────────
print(f"\n[5/5] {CV_FOLDS}-fold CV on Western data...")
X_west_top_all = X_west_sc[:, top_idx]
cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=42)

with timer("Cross-validation"):
    cv_auc = cross_val_score(svm_ref, X_west_top_all, y_west, cv=cv, scoring='roc_auc',
n_jobs=-1)
    cv_acc = cross_val_score(svm_ref, X_west_top_all, y_west, cv=cv, scoring='accuracy',
n_jobs=-1)
    cv_f1  = cross_val_score(svm_ref, X_west_top_all, y_west, cv=cv, scoring='f1_macro',
n_jobs=-1)

cv_results = pd.DataFrame({
    'fold': range(1, CV_FOLDS+1),
    'roc_auc': cv_auc, 'accuracy': cv_acc, 'f1_macro': cv_f1
})
cv_results.to_csv(os.path.join(RESULTS_DIR, 'crossval_western_results.csv'), index=False)
print(f"  CV ROC-AUC : {cv_auc.mean():.4f} ± {cv_auc.std():.4f}")
print(f"  CV Accuracy: {cv_acc.mean():.4f} ± {cv_acc.std():.4f}")
print(f"  CV Macro F1: {cv_f1.mean():.4f} ± {cv_f1.std():.4f}")

# ────────────────────────────────────────────────────────────────────────────
# 10. THRESHOLD TUNING  — vectorised, no Python loop
# ────────────────────────────────────────────────────────────────────────────
```

```
thresholds = np.round(np.arange(0.10, 0.91, 0.01), 2)
y_indi_mat = np.tile(y_indi, (len(thresholds), 1))         # (T, N)
y_pred_mat = (y_prob_ref[None, :] >= thresholds[:, None])   # (T, N)

tp = ((y_pred_mat == 1) & (y_indi_mat == 1)).sum(1)
fn = ((y_pred_mat == 0) & (y_indi_mat == 1)).sum(1)
tn = ((y_pred_mat == 0) & (y_indi_mat == 0)).sum(1)
fp = ((y_pred_mat == 1) & (y_indi_mat == 0)).sum(1)

tumor_recall  = np.where(tp+fn > 0, tp / (tp+fn), 0)
normal_recall = np.where(tn+fp > 0, tn / (tn+fp), 0)
accuracy      = (tp+tn) / len(y_indi)
precision     = np.where(tp+fp > 0, tp / (tp+fp), 0)
macro_f1      = 0.5 * (
    np.where(tp+fn > 0, 2*tp/(2*tp+fp+fn), 0) +
    np.where(tn+fp > 0, 2*tn/(2*tn+fn+fp), 0)
)

sweep_df = pd.DataFrame({
    'threshold': thresholds, 'accuracy': accuracy,
    'tumor_recall': tumor_recall, 'normal_recall': normal_recall, 'macro_f1': macro_f1
})
valid = sweep_df[sweep_df['tumor_recall'] >= 0.85]
best_t_row = (valid.loc[valid['normal_recall'].idxmax()]
              if len(valid) else sweep_df.loc[sweep_df['macro_f1'].idxmax()])
BEST_T = float(best_t_row['threshold'])
y_pred_tuned = (y_prob_ref >= BEST_T).astype(int)

sweep_df.to_csv(os.path.join(RESULTS_DIR, 'threshold_sweep.csv'), index=False)
```

```
print(f"\n  Recommended threshold: {BEST_T:.2f}"
     f"  tumor recall: {recall_score(y_indi, y_pred_tuned, pos_label=1):.4f}"
     f"  normal recall: {recall_score(y_indi, y_pred_tuned, pos_label=0):.4f}")

# 11. SUMMARIES

def row(model, y_pred, y_prob, thresh):
   return {
      'model': model, 'test_set': 'Indian', 'threshold': thresh,
      'accuracy':      accuracy_score(y_indi, y_pred),
      'roc_auc':       roc_auc_score(y_indi, y_prob),
      'tumor_recall':  recall_score(y_indi, y_pred, pos_label=1),
      'normal_recall': recall_score(y_indi, y_pred, pos_label=0),
      'macro_f1':      f1_score(y_indi, y_pred, average='macro'),
   }

summary = pd.DataFrame([
   row('Baseline LinearSVC (all genes)',            y_pred_base,  y_prob_base, 0.50),
   row(f'Refined SVM RBF top-{TOP_N_GENES} (t=0.50)', y_pred_ref,  y_prob_ref,  0.50),
   row(f'Refined SVM RBF top-{TOP_N_GENES} (t={BEST_T:.2f} tuned)', y_pred_tuned,
y_prob_ref, BEST_T),
])
summary.to_csv(os.path.join(RESULTS_DIR, 'model_summary.csv'), index=False)
print("\nModel summary:")
print(summary.to_string(index=False))

top30_rf   = set(rf_importance.head(30).index)
top30_shap = set(top_genes[:min(30, len(top_genes))])
overlap    = top30_rf & top30_shap
pd.DataFrame({'gene': sorted(overlap), 'in_rf_top30': True, 'in_shap_top30': True}
```

```
).to_csv(os.path.join(RESULTS_DIR, 'biomarker_overlap_RF_SHAP.csv'), index=False)
print(f"Biomarker overlap (RF top-30 ∩ SHAP top-30): {len(overlap)} genes")


# 12. PLOTS

print("\nGenerating plots...")

# Plot 1: Confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, y_pred, title, cmap in zip(
    axes,
    [y_pred_base, y_pred_ref, y_pred_tuned],
    ['Baseline LinearSVC\n(all genes)',
     f'Refined SVM\n(RBF, top-{TOP_N_GENES}, t=0.50)',
     f'Refined SVM\n(RBF, top-{TOP_N_GENES}, t={BEST_T:.2f} tuned)'],
    ['Blues', 'Purples', 'Greens']
):
    cm = confusion_matrix(y_indi, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap=cmap, ax=ax,
                xticklabels=['Normal','Tumor'], yticklabels=['Normal','Tumor'],
                annot_kws={'size': 13})
    ax.set_title(title, fontweight='bold', fontsize=9)
    ax.set_xlabel('Predicted'); ax.set_ylabel('Actual')
plt.suptitle('WestIndiSVM — Confusion Matrices (Indian Test Set)', fontsize=12,
fontweight='bold')
plt.tight_layout(); savefig('plot1_confusion_matrices.png')

# Plot 2: ROC curves
fig, ax = plt.subplots(figsize=(7, 5))
```

```
for y_prob, label, color in [
    (y_prob_base, f'Baseline (AUC={roc_auc_score(y_indi,y_prob_base):.3f})', BLUE),
    (y_prob_ref,  f'Refined  (AUC={roc_auc_score(y_indi,y_prob_ref):.3f})',  RED),
]:
    fpr, tpr, _ = roc_curve(y_indi, y_prob)
    ax.plot(fpr, tpr, lw=2, color=color, label=label)
ax.plot([0,1],[0,1],'--',color='gray',lw=1)
ax.set_xlabel('FPR'); ax.set_ylabel('TPR')
ax.set_title('WestIndiSVM — ROC Curves (Indian Test Set)', fontweight='bold')
ax.legend(); ax.grid(alpha=0.3); plt.tight_layout(); savefig('plot2_roc_curves.png')

# Plot 3: Precision-Recall
fig, ax = plt.subplots(figsize=(7, 5))
for y_prob, label, color in [
    (y_prob_base, f'Baseline (AP={average_precision_score(y_indi,y_prob_base):.3f})',
BLUE),
    (y_prob_ref,  f'Refined  (AP={average_precision_score(y_indi,y_prob_ref):.3f})',  RED),
]:
    prec, rec, _ = precision_recall_curve(y_indi, y_prob)
    ax.plot(rec, prec, lw=2, color=color, label=label)
ax.set_xlabel('Recall'); ax.set_ylabel('Precision')
ax.set_title('WestIndiSVM — Precision-Recall (Indian Test Set)', fontweight='bold')
ax.legend(); ax.grid(alpha=0.3); plt.tight_layout(); savefig('plot3_precision_recall.png')

# Plot 4: SHAP biomarkers
fig, ax = plt.subplots(figsize=(9, 8))
top_shap_s = shap_importance.head(TOP_N_GENES)
colors = [RED if i < 5 else BLUE if i < 10 else GREEN for i in range(TOP_N_GENES)]
ax.barh(range(TOP_N_GENES), top_shap_s.values[::-1], color=colors[::-1], alpha=0.85)
```

```
ax.set_yticks(range(TOP_N_GENES)); ax.set_yticklabels(top_shap_s.index[::-1],
fontsize=8)
ax.set_xlabel('Mean |SHAP value|')
ax.set_title(f'WestIndiSVM — Top {TOP_N_GENES} SHAP Biomarkers', fontweight='bold')
ax.grid(axis='x', alpha=0.3); plt.tight_layout(); savefig('plot4_shap_biomarkers.png')

# Plot 5: RF importance
fig, ax = plt.subplots(figsize=(9, 8))
top_rf_s = rf_importance.head(TOP_N_GENES)
ax.barh(range(TOP_N_GENES), top_rf_s.values[::-1], color=PURPLE, alpha=0.85)
ax.set_yticks(range(TOP_N_GENES)); ax.set_yticklabels(top_rf_s.index[::-1], fontsize=8)
ax.set_xlabel('RF Feature Importance (Gini)')
ax.set_title(f'WestIndiSVM — Top {TOP_N_GENES} RF Biomarkers', fontweight='bold')
ax.grid(axis='x', alpha=0.3); plt.tight_layout(); savefig('plot5_rf_biomarkers.png')

# Plot 6: Cross-validation
fig, ax = plt.subplots(figsize=(8, 5))
x = np.arange(CV_FOLDS); w = 0.25
b1 = ax.bar(x-w, cv_auc, w, label='ROC-AUC', color=BLUE, alpha=0.85)
b2 = ax.bar(x,   cv_acc, w, label='Accuracy', color=GREEN, alpha=0.85)
b3 = ax.bar(x+w, cv_f1,  w, label='Macro F1', color=RED, alpha=0.85)
for bars in [b1, b2, b3]:
    for bar in bars:
        ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.005,
                f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=7)
ax.set_xticks(x); ax.set_xticklabels([f'Fold {i+1}' for i in range(CV_FOLDS)])
ax.set_ylim(0, 1.12); ax.set_ylabel('Score')
ax.set_title(f'WestIndiSVM — {CV_FOLDS}-Fold CV (Western)
AUC={cv_auc.mean():.3f}±{cv_auc.std():.3f}',
             fontweight='bold')
```

```
ax.legend(); ax.grid(axis='y', alpha=0.3); plt.tight_layout();
savefig('plot6_cv_western.png')


# Plot 7: Threshold sweep

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(sweep_df['threshold'], sweep_df['tumor_recall'],  color=RED,   lw=2.5,
label='Tumor recall')

ax.plot(sweep_df['threshold'], sweep_df['normal_recall'], color=BLUE,  lw=2.5,
label='Normal recall')

ax.plot(sweep_df['threshold'], sweep_df['macro_f1'],      color=GREEN, lw=1.8, ls='--',
label='Macro F1')

ax.axvline(0.50,   color='gray', lw=1.2, ls='--', alpha=0.7, label='Default (0.50)')

ax.axvline(BEST_T, color=PURPLE, lw=2,            label=f'Recommended ({BEST_T:.2f})')

ax.set_xlabel('Decision Threshold'); ax.set_ylabel('Score')

ax.set_title('WestIndiSVM — Threshold Tuning (Indian Test Set)', fontweight='bold')

ax.legend(); ax.set_xlim(0.10, 0.90); ax.set_ylim(0, 1.05); ax.grid(alpha=0.3)

plt.tight_layout(); savefig('plot7_threshold_tuning.png')


# Plot 8: SHAP beeswarm

plt.figure(figsize=(10, 8))

shap.summary_plot(

   shap_tumor[:, top_idx],

   X_indi_sc[:SHAP_N_SAMPLES, :][:, top_idx],

   feature_names=top_genes, show=False, plot_size=None

)

plt.title(f'WestIndiSVM — SHAP Beeswarm (top-{TOP_N_GENES} genes, Indian test)',
fontweight='bold')

plt.tight_layout(); savefig('plot8_shap_beeswarm.png')


# Plot 9: Score distribution

fig, ax = plt.subplots(figsize=(9, 4))
```

```
ax.hist(y_prob_ref[y_indi==0], bins=30, alpha=0.65, color=BLUE, label='Normal',
density=True)

ax.hist(y_prob_ref[y_indi==1], bins=30, alpha=0.65, color=RED,  label='Tumor',
density=True)

ax.axvline(0.50,   color='gray', lw=1.5, ls='--', label='Default (0.50)')

ax.axvline(BEST_T, color=PURPLE, lw=2,           label=f'Recommended ({BEST_T:.2f})')

ax.set_xlabel('Predicted Tumor Probability'); ax.set_ylabel('Density')

ax.set_title('WestIndiSVM — Score Distribution (Indian Test Set)', fontweight='bold')

ax.legend(); ax.grid(alpha=0.3); plt.tight_layout(); savefig('plot9_score_distribution.png')


# Plot 10: Biomarker overlap

top30_rf_list   = rf_importance.head(30).index.tolist()

top30_shap_list = shap_importance.head(30).index.tolist()

only_rf   = [g for g in top30_rf_list   if g not in top30_shap_list]

only_shap = [g for g in top30_shap_list if g not in top30_rf_list]

in_both   = sorted(overlap)

fig, ax = plt.subplots(figsize=(8, 4))

ax.barh(['RF only','SHAP only','In both'],

       [len(only_rf), len(only_shap), len(in_both)],

       color=[BLUE, RED, GREEN], alpha=0.85)

for i, v in enumerate([len(only_rf), len(only_shap), len(in_both)]):

   ax.text(v+0.2, i, str(v), va='center', fontweight='bold')

ax.set_xlabel('Number of Genes')

ax.set_title('WestIndiSVM — Biomarker Overlap (RF vs SHAP Top-30)', fontweight='bold')

ax.grid(axis='x', alpha=0.3); plt.tight_layout(); savefig('plot10_biomarker_overlap.png')



print("\n" + "=" * 65)

print(f"DONE — results saved to: {RESULTS_DIR}")

print("=" * 65)
```